Ch 01: Operating System Interfaces

xv6 a simple Unix-like teaching Operating System

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2026

EAST TEXAS A&M

EAST TEXAS A&M

Ch 01: System Call Interface, Processes and Memory

xv6 a simple Unix-like teaching Operating System

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2026

Operating System Interfaces

OS Purpose

The job of an operating system is to share a computer among multiple programs and to provide a more useful set of services than the hardware alone supports.

The Operating System:

- Provides abstractions of low-level hardware
- Manages resources, shares among multiple programs
- Allows programs to interact

An OS provides services through an interface (system call API)

- Simple and narrow, easy to use
- Powerful, supports complex features
- **Provide basic mechanisms that combine generally**
  The *Unix* way

xv6 Operating System

- (Re)implementation of Unix V6 History of Unix (Wikipedia, 2026)
- We are using a RISC V port for this class
- **kernel**: a special program that provides services to running programs
- **process**: a running program that uses OS kernel services
  - allocated memory containing *instructions* to be executed
  - *data* for the instructions to perform computations with
  - *stack* organizes program's procedure calls.

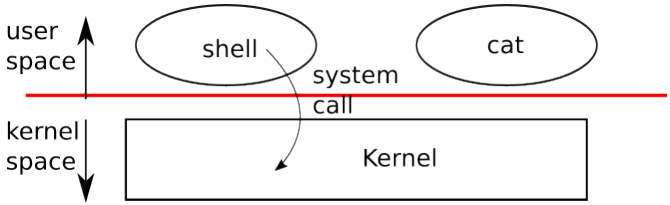

Figure 1: A kernel and two user processes (Cox et al., 2026, pg.10)

EAST TEXAS A&M

xv6 System Calls

- **system call**: the system interface of the OS kernel
  - **kernel space**: privileged mode supported by hardware protection mechanisms
  - **user space** normal program instruction execution

xv6 System Calls

Table 1: xv6 system calls. If not otherwise stated, these calls return 0 for no error, and -1 if there's an error.

System Call	Description
`int fork()`	Create a process, return child's `PID`.
`int exit(int status)`	Terminate the current process; status reported to `wait()`. No return.
`int wait(int *status)`	Wait for a child to exit; exit status in `*status`; returns child PID.
`int kill(int pid)`	Terminate process `PID`. Returns 0, or -1 for error.
`int getpid()`	Return the current process's `PID`.
`int pause(int n)`	Pause for `n` clock ticks.
`int exec(char *file, char *argv[])`	Load a file and execute it with arguments; only returns if error.
`char *sbrk(int n)`	Grow process's memory by `n` zero bytes. Returns start of new memory.
`int open(char *file, int flags)`	Open a file; flags indicate read/write; returns an `fd` (file descriptor).
`int write(int fd, char *buf, int n)`	Write `n` bytes from `buf` to file descriptor `fd`; returns `n`.
`int read(int fd, char *buf, int n)`	Read `n` bytes into `buf`; returns number read; or 0 if end of file.
`int close(int fd)`	Release open file `fd`.
`int dup(int fd)`	Return a new file descriptor referring to the same file as `fd`.
`int pipe(int p[])`	Create a pipe, put read/write file descriptors in `p[0]` and `p[1]`.
`int chdir(char *dir)`	Change the current directory.
`int mkdir(char *dir)`	Create a new directory.
`int mknod(char *file, int, int)`	Create a device file.
`int fstat(int fd, struct stat *st)`	Place info about an open file into `*st`.
`int link(char *file1, char *file2)`	Create another name (`file2`) for the file `file1`.
`int unlink(char *file)`	Remove a file.

EAST TEXAS A&M

Processes and Memory (1 of 3)

- An xv6 processes consists of
  - user-space memory (instructions, data and stack)
  - per-process state private to the kernel (process table)
    - kernel associates a process identifier `PID` with each process
- xv6 **time-shares** processes: it transparently switches the available CPUs among set of processes waiting to execute.

Processes and Memory (2 of 3)

- A process may create a new process by calling **`fork()`** system call
  - `fork()` gives new process exact copy of calling process's memory
  - returns twice, `PID` 0 in child and non-zero in parent
- **`exit()`** system call causes process to stop executing
  - release resource back to kernel, such as memory and open files
- **`wait()`** system call returns `PID` of exited child or waits for one to exit
- The **`exec()`** system call replaces process's memory with a new image
  - Loaded from file

EAST TEXAS A&M

Processes and Memory (3 of 3)

fork() system call (ex1-fork.c)

```c
int pid = fork();

if (pid > 0) {
  printf("parent: child=%d\n", pid);
  pid = wait((int *) 0);
  printf("child %d is done\n", pid);
} else if (pid == 0) {
  printf("child: exiting\n");
  exit(0);
} else {
  printf("fork error\n");
}
```

exec() system call (ex2-exec.c)

```c
char *argv[3];
argv[0] = "echo";
argv[1] = "hello";
argv[2] = 0;
// does not return unless error
exec("echo", argv);
printf("exec error\n");
```

EAST TEXAS A&M

Summary

- Operating Systems
  - provide simplified abstract interfaces for low-level hardware
  - manage and allocate resources to share among multiple programs
  - **system call** the system interface of the OS kernel
  - **kernel**: provides services for running processes
    - **kernel space** privileged mode supported by hardware protection mechanisms
  - **process**: a running program that uses OS kernel services
    - **user space** normal unprivileged execution of instructions

EAST TEXAS A&M

Ch 01: Input/Output and File Descriptors

xv6 a simple Unix-like teaching Operating System

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2026

I/O and File Descriptors

File Descriptor `fd`

An integer representing a kernel-managed object that a process may read from or write to

- Can be obtained by
  - Opening a file, directory or device
  - Duplicating an existing descriptor
  - Creating a pipe
- By convention:
  - `fd` 0 reads from **standard input** (keyboard)
  - `fd` 1 writes to **standard output** (terminal)
  - `fd` 2 writes to **standard error** (terminal)

- **`read(fd, buf, n)`** system call
  - reads at most `n` bytes from file descriptor `fd`
  - copies bytes into `buf`
  - returns `n` the actual number of bytes read

- **`write(fd, buf, n)`** system call
  - Writes `n` bytes from `buf` to the file descriptor `fd`
  - returns `n` number of bytes successfully written

I/O and File Descriptors cat system command

```
char buf[512];
int n;
for (;;) {
  n = read(0, buf, sizeof buf);
  if (n == 0)
    break;
  if (n < 0) {
    fprintf(2, "read error\n");
    exit(1);
  }
  if (write(1, buf, n) != n) {
    fprintf(2, "write error\n");
    exit(1);
  }
}
```

- Doesn't know if `fd` 0 is reading from a file, console or a pipe
- Doesn't know where `fd` 1 is writing to either
- `fd` is an abstraction of a byte stream

I/O Redirection and open()

- `close()` system call releases file descriptor for reuse
- `fork()` copies paren'ts file descriptor table along with memory
  - so child starts with exactly same open files as parament
  - `exec()` replaces calling processes memory but preserves its file table
- I/O redirection by shell is thus accomplished similar to this example
  - `cat < input.txt`
- `open()` system call opens a new file descriptor
  - second argument set of flags as bits
    - O_RDONLY for reading
    - O_WRONGLY for writing
    - O_RDWR for both read and write
    - O_CREATE to create a file if it doesn't exist
    - O_TRUNC to overwrite existing file (truncate to 0 length)

(ex4-ioredirect.c)

```
char *argv[2];

argv[0] = "cat";
argv[1] = 0;
if (fork() == 0) {
    close(0);
    open("input.txt", O_RDONLY);
    exec("cat", argv);
}
```


EAST TEXAS A&M

Pipes

Pipe

A **pipe** is a small kernel buffer exposed to processes as a pair of file descriptors, one for reading and one for writing.

Writing data to one end of the pipe makes the data available for reading from the other end of the pipe.

Advantages over temporary files:

1. automatically clean themselves up
2. can pass arbitrarily long streams of data
3. parallel execution of pipeline stages

(ex5-pipe.c)

```
int p[2];
char *argv[2];

argv[0] = "wc";
argv[1] = 0;

pipe(p);

if (fork() == 0) {
    close(0);
    dup(p[0]);
    close(p[0]);
    close(p[1]);
    exec("wc", argv);
}
else {
    close(p[0]);
    write(p[1], "hello world\n", 12);
    close(p[1]);
}
```

EAST TEXAS A&M

Summary

- File descriptors are a powerful abstraction
  - They hide details of what they are connected to: a file, a device, a pipe
- Pipes are small kernel buffers exposed to processes as a pair of file descriptors for reading / writing from / to

EAST TEXAS A&M

Ch 01: File System

xv6 a simple Unix-like teaching Operating System

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2026

File System

The xv6 file system is a basic Unix file system

- **Data files** are uninterpreted byte arrays (interpreted by programs only, not OS)
- **Directories** contained named references to data files and directories
  - directories form a tree
  - special directory called the **root** / (also the path separator, can be confusing)
  - Paths not starting at root / are evaluated **relative** to the **current directory**
  - Current directory can be changed using the `chdir()` system call

These are equivalent:

```
chdir("/a");
chdir("b");
open("c", O_RDONLY);

open("/a/b/c", O_RDONLY);
```

File System Calls

- **`mkdir()`** creates a new directory
- **`open()`** with `O_CREATE` flag creates a new data file
- **`mknod()`** creates a new device file

```
mkdir("/dir");
fd = open("/dir/file", O_CREATE|O_WRONLY);
close(fd);
mknod("/console", 1, 1);
```

Device Files

- `mknod()` creates a special file that refers to a device
  - major and minor device number (arguments to `mknod()`)
  - identifies a **kernel** device
    - When a process opens a device file, `read()` and `write()` system calls routed to the device driver associated with major/minor device number

EAST TEXAS A&M

File Names

- A file name is distinct from the file itself
  - Same underlying file, **inode**
  - can have multiple names **links**
  - Each link consists of an entry in a directory, entry contains a file name and reference to an inode.
  - **inode** holds *metadata* about a file
    - type (directory, device, data)
    - length (size)
    - location on disk (blocks)
- `fstat()` system call retrieves information from the inode that a file descriptor refers to.
  - `struct stat` data structure in `stat.h`
- `link()` system call creates another file system name referring to the same inode as an existing file.
- `unlink()` system call removes a name from the file system.
  - When number of links to inode becomes 0, content is freed.

Summary

- File system in Unix like system contain
  - **data files**: arrays / streams of bytes
  - **directories**: organize file system structure into tree hierarchy

EAST TEXAS A&M

References

- Cox, R., Kaashoek, F., & Morris, R. (2026). *Xv6: A simple, unix-like teaching operating system*. Retrieved September 2, 2025, from https://github.com/mit-pdos/xv6-riscv-book
- Wikipedia. (2026). *History of unix*. Retrieved August 14, 2026, from https://en.wikipedia.org/wiki/History_of_Unix

EAST TEXAS A&M